Student 23 **Padmanabhuni Revanth Sai** 192525451RD 2 / 2 submitted

Q1. Student Grade Book Application

CODE

```
students = {}

def grade(avg):
    if avg ≥ 90:
        return "O"
    elif avg ≥ 80:
        return "A+"
    elif avg ≥ 70:
        return "A"
    elif avg ≥ 60:
        return "B+"
    elif avg ≥ 50:
        return "B"
    else:
        return "F"

n = 5

for i in range(n):
    name = input("Enter student name: ")
    marks = list(map(int, input("Enter 5 subject marks: ").split()))
    students[name] = marks

result = {}

for name, marks in students.items():
    total = sum(marks)
    average = total / 5
    result[name] = (total, average, grade(average))

topper = max(result.items(), key=lambda x: x[1][1])

subject_means = []
for i in range(5):
    s = 0
    for marks in students.values():
        s += marks[i]
    subject_means.append(s / n)

highest_subject = subject_means.index(max(subject_means)) + 1

sorted_result = sorted(result.items(), key=lambda x: x[1][1], reverse=True)

print("\nName\tTotal\tAverage\tGrade")
for name, data in sorted_result:
    print(name, "\t", data[0], "\t", round(data[1], 2), "\t", data[2])

print("\nClass Topper:", topper[0])
print("Highest Overall Mean: Subject", highest_subject)
```

INPUT

```
Rahul
90 80 85 88 91
Anu
78 82 80 75 79
Sai
95 96 94 93 97
Priya
65 70 68 72 66
Ravi
50 55 60 58 52
```

OUTPUT

```
(no output)
```


SIMATS Engineering • CSE • CSA0804 — Python Programming • 05-08-2026 • Category: Application Based Questions • Student Submissions Report

Q2. Debugging and Rewriting

CODE

```
accounts = {}

def create_account(acc_no, name, balance):
    accounts[acc_no] = {
        'name': name,
        'balance': balance,
        'txns': []
    }

def deposit(acc_no, amount):
    if acc_no in accounts:
        accounts[acc_no]['balance'] += amount
        accounts[acc_no]['txns'].append(('Deposit', amount))

def withdraw(acc_no, amount):
    if acc_no in accounts:
        if accounts[acc_no]['balance'] ≥ amount:
            accounts[acc_no]['balance'] -= amount
            accounts[acc_no]['txns'].append(('Withdraw', amount))
        else:
            print("Insufficient balance")

def mini_statement(acc_no):
    if acc_no in accounts:
        print("Mini Statement")
        for t in accounts[acc_no]['txns'][-5:]:
            print(t)

create_account(1001, "Ravi", 10000)
deposit(1001, 5000)
withdraw(1001, 2000)
mini_statement(1001)
print("Balance:", accounts[1001]['balance'])
```

OUTPUT

```
(no output)
```

```
    elif avg ≥ 50:
        return "B"
    else:
        return "F"

n = 5

for i in range(n):
    name = input("Enter student name: ")
    marks = list(map(int, input("Enter 5 subject marks: ").split()))
    students[name] = marks

result = {}

for name, marks in students.items():
    total = sum(marks)
    average = total / 5
    result[name] = (total, average, grade(average))

topper = max(result.items(), key=lambda x: x[1][1])

subject_means = []
for i in range(5):
    s = 0
    for marks in students.values():
        s += marks[i]
    subject_means.append(s / n)

highest_subject = subject_means.index(max(subject_means)) + 1

sorted_result = sorted(result.items(), key=lambda x: x[1][1], reverse=True)

print("\nName\tTotal\tAverage\tGrade")
for name, data in sorted_result:
    print(name, "\t", data[0], "\t", round(data[1], 2), "\t", data[2])

print("\nClass Topper:", topper[0])
print("Highest Overall Mean: Subject", highest_subject)
```

INPUT

```
Rahul
90 80 85 88 91
Anu
78 82 80 75 79
Sai
95 96 94 93 97
Priya
65 70 68 72 66
Ravi
50 55 60 58 52
```

OUTPUT

```
(no output)
```


SIMATS Engineering • CSE • CSA0804 — Python Programming • 05-08-2026 • Category: Application Based Questions • Student Submissions Report

Q2. Debugging and Rewriting

CODE

```
accounts = {}

def create_account(acc_no, name, balance):
    accounts[acc_no] = {
        'name': name,
        'balance': balance,
        'txns': []
    }

def deposit(acc_no, amount):
    if acc_no in accounts:
        accounts[acc_no]['balance'] += amount
        accounts[acc_no]['txns'].append(('Deposit', amount))

def withdraw(acc_no, amount):
    if acc_no in accounts:
        if accounts[acc_no]['balance'] ≥ amount:
            accounts[acc_no]['balance'] -= amount
            accounts[acc_no]['txns'].append(('Withdraw', amount))
        else:
            print("Insufficient balance")

def mini_statement(acc_no):
    if acc_no in accounts:
        print("Mini Statement")
        for t in accounts[acc_no]['txns'][-5:]:
            print(t)

create_account(1001, "Ravi", 10000)
deposit(1001, 5000)
withdraw(1001, 2000)
mini_statement(1001)
print("Balance:", accounts[1001]['balance'])
```

OUTPUT